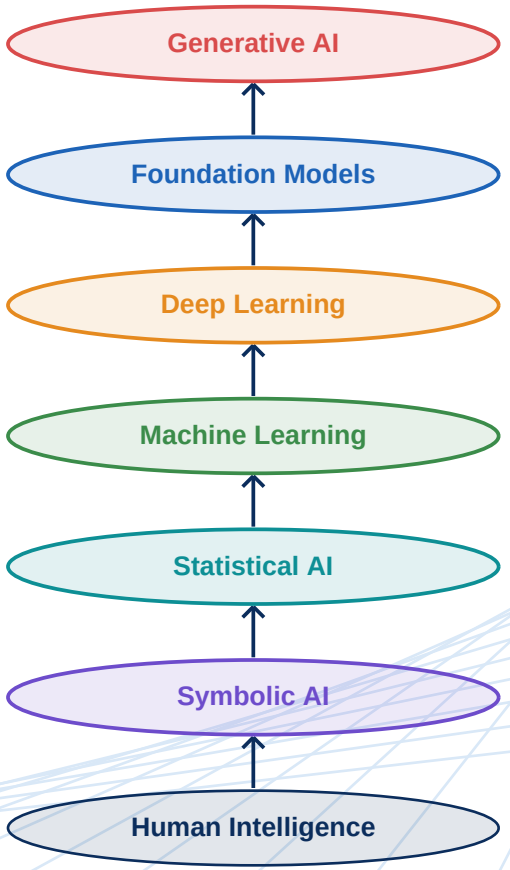
GENERATIVE ARTIFICIAL INTELLIGENCE

MODULE 1

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

From Symbolic Reasoning to Machine Learning



TODAY'S TOPICS

- Evolution of Artificial Intelligence
- Symbolic AI vs Statistical AI
- Machine Learning: Classification vs Regression
- Generative Models vs Discriminative Models

B.Tech III & IV Year

Module Overview

A compact roadmap for today's lecture

1. **Evolution of Artificial Intelligence**
2. **Symbolic AI vs Statistical AI**
3. **Machine Learning Foundations**
4. **Classification vs Regression**
5. **Generative vs Discriminative Models**

Learning Outcomes

By the end of this module, students should be able to:

- Explain how Artificial Intelligence evolved through multiple technological waves.
- Differentiate rule-based Symbolic AI from probabilistic and data-driven Statistical AI.
- Explain the machine-learning view: learn a function from data rather than hand-code every rule.
- Distinguish classification and regression by target type, loss and prediction.
- Explain the probabilistic distinction between generative and discriminative models.

CORE LEARNING PATH

Rules → Probability → Learning → Prediction → Generation

Table of Contents

Module 1 - Foundations of Artificial Intelligence

PART I

Evolution of Artificial Intelligence

- What is intelligence?
- What is Artificial Intelligence?
- Turing's question and the Dartmouth workshop
- Symbolic AI and expert systems

+ additional worked examples and derivations

PART II

Symbolic AI vs Statistical AI

- Knowledge representation and rules
- Search and inference
- Uncertainty and Bayesian thinking
- Rule-based vs probabilistic reasoning

PART III

Machine Learning

- Data, samples, features and labels
- Supervised / unsupervised / reinforcement learning
- Training, validation and testing

+ additional worked examples and derivations

PART IV

Classification vs Regression

- Regression and linear vs nonlinear models
- Four views of linear regression
- Classification, logistic regression, SVM and trees

PART V

Generative vs Discriminative Models

- Bayes and Naive Bayes
- Markov chains, HMMs and graphical models
- Generative vs discriminative probability models

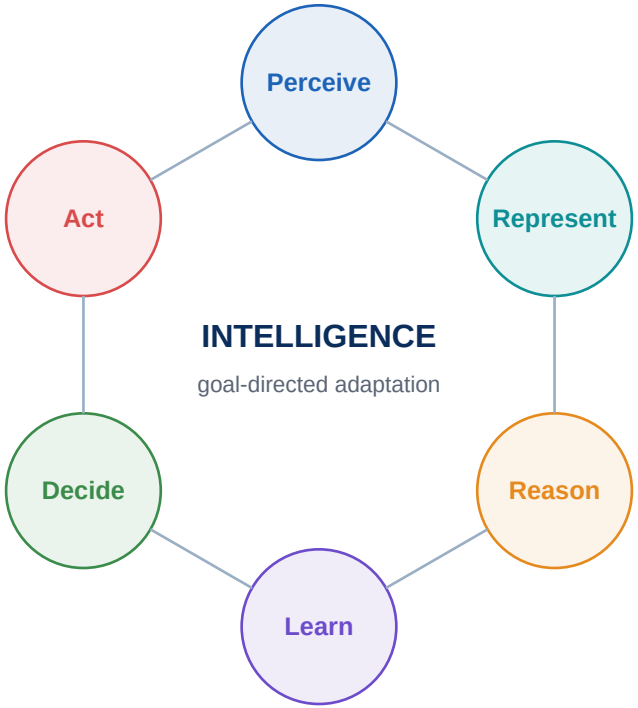
+ additional worked examples and derivations

Part I - Evolution of Artificial Intelligence

Start with the question: what do we mean by intelligence?

1. What makes an intelligent system intelligent?

A useful operational view is a closed loop: perceive the world, build an internal representation, reason, learn, decide and act.



2. What is Artificial Intelligence?

Artificial Intelligence studies computational systems that perform tasks associated with intelligent behavior.

Across history, different AI eras emphasized different mechanisms: rules, probability, optimization, learned representations and scale.

$$a = f(x)$$

observation x -> model f -> action a

EXAMPLE: SELF-DRIVING CAR

- Perceive - cameras and lidar detect lanes, vehicles and pedestrians.
- Represent - maintain an internal state of the road.
- Reason - predict possible movements of nearby vehicles.
- Decide & Act - select steering, braking and acceleration.
- Learn - improve models using data and feedback.

THINK: Is a calculator intelligent? A chess engine? A recommendation system? Which criterion changes your answer?

Evolution of Artificial Intelligence

Each wave solved some limitations and revealed new ones.

1950 — Turing
Behavioral test for machine intelligence

1956 — Dartmouth
AI becomes a distinct research field

1960s — Symbolic AI
Logic, rules, search and knowledge bases

1970s — Expert Systems
DENDRAL, MYCIN; narrow expert reasoning

1980s — AI Winter
Expectations exceed available data and compute

1990s — Statistical ML
Regression, SVMs, trees and Bayes learn from data

2012 — Deep Learning
AlexNet; GPUs + big data + learned representations

2017 — Transformers
Attention enables scalable sequence modeling

2022 — Foundation Models
Broad pretrained models adapted across tasks

2025+ — Agentic AI
Models + tools + retrieval + memory + planning

THE BIG HISTORICAL ARC

Rules — human-coded knowledge → **Probability** — uncertainty → **Learning** — patterns from data → **Representation** — features learned → **Scale** — broad pretraining → **Agency** — tools and action

KEY TAKEAWAY: Modern AI combines rules, probability, learning, representation and tool-based action; one paradigm did not simply erase the others.

PART 2

Symbolic AI vs Statistical AI

From hand-coded rules to probabilistic learning



IF condition THEN conclusion

P(hypothesis | evidence)

KEY QUESTION

Why did AI move from manually written knowledge to models that estimate patterns from data?

Refs: Russell & Norvig Ch.1-2; Bishop Ch.1

What is Symbolic AI?

Intelligence as explicit symbols, rules and logical inference

Symbolic AI represents the world using human-readable symbols such as objects, relations, rules and logical statements. The system reasons by manipulating these symbols using inference procedures.

Symbolic view

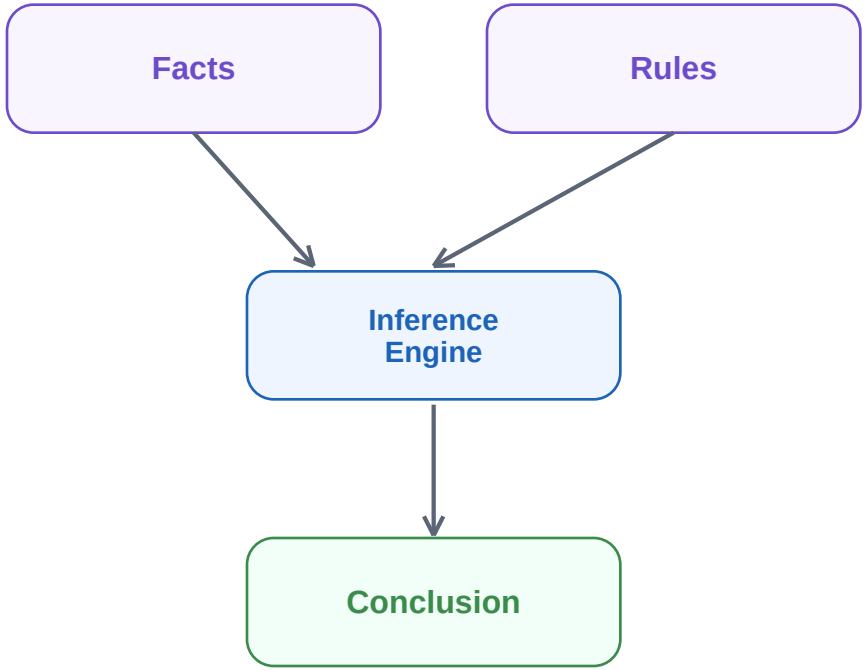
Knowledge + Logic -> Inference

EXAMPLE RULE

IF fever AND cough

THEN possible_flu

Strong when rules are complete; brittle when symptoms are noisy.



Book basis: Symbolic AI uses logic, rules, knowledge bases and search

Knowledge Representation

How do we encode facts about the world?

A symbolic system first needs a language for representing knowledge. The representation determines what can be reasoned about, how efficiently inference can run and how explainable the system is.

Logic

```
Parent(Alice,Bob)
Human(Socrates)
Human(x) -> Mortal(x)
```

Semantic Network

```
Disease
  | has_symptom
Fever
  | suggests
Diagnosis
```

Production Rules

```
IF fever AND cough
THEN possible_flu
confidence = high
```

Logical implication

Human(Socrates) and Human(x) -> Mortal(x) => Mortal(Socrates)

Refs: Russell & Norvig Ch.7-8

Rule-Based Systems

Programs that reason using IF-THEN rules

A rule-based system applies production rules to known facts. When the premise of a rule is satisfied, the system fires the rule and adds a new conclusion to its working memory.




```
working_memory = {fever, cough}
for rule in knowledge_base:
    if rule.premise is true:
        add rule.conclusion
```

TEACHING INSIGHT

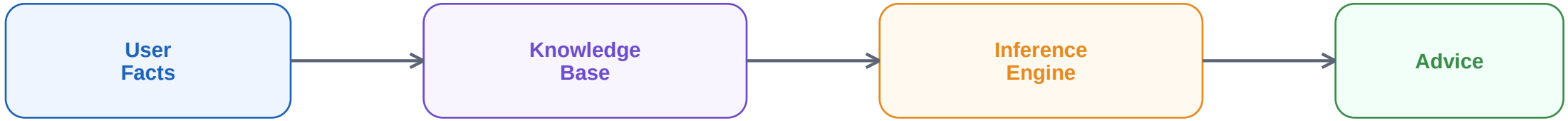
Rule systems are attractive because every conclusion can be traced back to a rule. This makes them interpretable, but also dependent on human-written knowledge.

Book basis: rule example IF fever AND cough THEN possible_flu

Expert Systems

Symbolic AI at industrial scale

Expert systems attempted to capture specialist knowledge from human experts and reproduce expert reasoning in narrow domains such as medical diagnosis, chemistry and industrial troubleshooting.



DENDRAL

Assisted chemists in molecular structure identification.

MYCIN

Diagnosed bacterial infections using medical expert rules.

Key lesson: expert systems worked well in narrow domains, but were hard to scale and maintain.

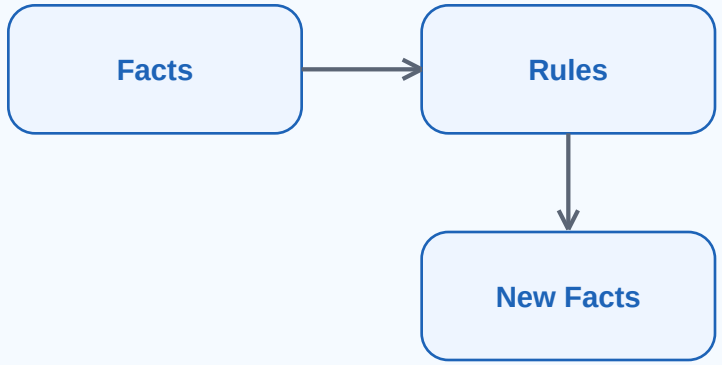
Book basis: DENDRAL and MYCIN are classic expert systems

Search and Inference

Forward chaining vs backward chaining

Forward Chaining

Data-driven reasoning: start from known facts and repeatedly apply rules until a conclusion is reached.

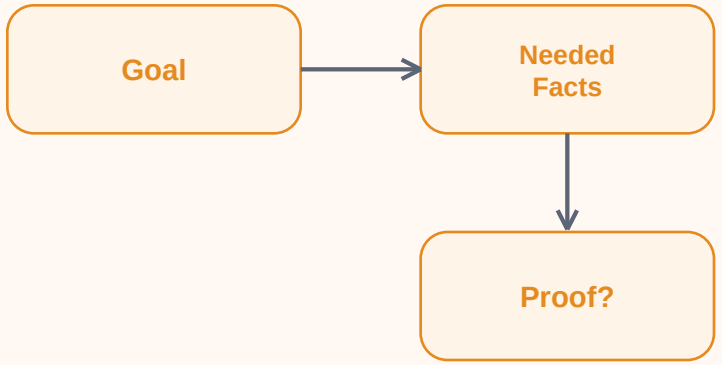


Use when:

- Many possible goals
- Stream of incoming facts
- Monitoring and diagnosis

Backward Chaining

Goal-driven reasoning: start from a hypothesis and ask what facts must be true to prove it.



Use when:

- Specific hypothesis to prove
- Interactive troubleshooting
- Medical consultation systems

Refs: Russell & Norvig Ch.7

Why Symbolic AI Struggled

The real world is noisy, incomplete and constantly changing

More rules do not automatically mean more intelligence

Knowledge acquisition bottleneck

Experts must manually encode thousands of rules.

Brittleness

Unseen cases may fail because no rule matches.

Uncertainty

Hard rules struggle with noisy or incomplete evidence.

Scalability

Rule interactions become difficult to test and maintain.

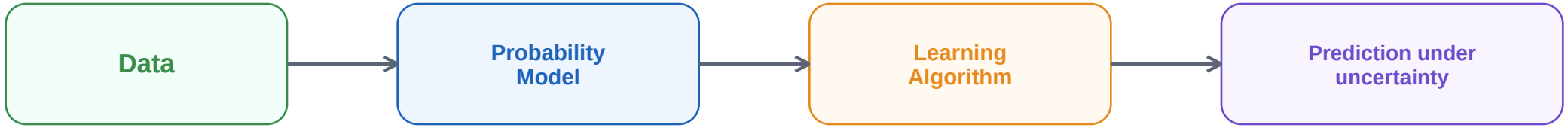
Transition: from 'What rules should we write?' to 'What patterns can we learn from data?'

Book basis: limitations include bottleneck, scalability, uncertainty and maintenance

What is Statistical AI?

Intelligence as probabilistic pattern estimation

Statistical AI treats intelligence as inference under uncertainty. Instead of manually enumerating every rule, it uses data to estimate probabilities, parameters and decision boundaries.



Core statistical question

$$P(\text{hypothesis} \mid \text{evidence})$$

How should belief change after observing data?

Book basis: statistical AI estimates patterns from observations and represents uncertainty

Bayesian Thinking

Updating belief using evidence

Bayes' theorem

$$P(H \mid E) = P(E \mid H)\, P(H) / P(E)$$

Prior

P(H)

belief before evidence

Likelihood

P(E | H)

how expected evidence is

Evidence

P(E)

overall probability of evidence

Posterior

P(H | E)

updated belief after evidence

WORKED EXAMPLE: RARE DISEASE TEST

Given: P(D)=0.01, P(+|D)=0.95, P(+|not D)=0.05

P(+)=0.95(0.01)+0.05(0.99)=0.059, so P(D|+)=0.0095/0.059=0.161

Posterior = 16.1%, not 95%

Refs: Bishop Ch.1; Murphy Ch.1

Symbolic vs Statistical AI

Why Machine Learning became the next major step

This slide summarizes the two paradigms side by side and highlights why data-driven learning became dominant.

Dimension	Symbolic AI	Statistical AI
Knowledge source	Human-written rules	Observed data
Reasoning style	Logic and search	Probability and estimation
Uncertainty	Difficult to represent	Central design principle
Strength	Interpretable conclusions	Robust pattern learning
Weakness	Brittle; hard to scale	Needs representative data
Modern role	Rules, constraints, knowledge graphs	ML, Bayes and deep learning

TRANSITION TO MACHINE LEARNING

Rules -> Knowledge | Data -> Learning | Learning -> Prediction

Book basis: statistical learning shifted from writing rules to learning patterns from data

PART 3

Machine Learning Foundations

High-level ML view before detailed algorithms



ROLE OF PART 3

This part introduces the ML + GenAI model universe, datasets, matrix view, pipeline, loss/cost/risk and generalization.

Detailed regression/classification algorithms move to Part 4

What is Machine Learning?

Learning patterns from examples

Machine learning is a way to build systems that improve on a task using experience. Instead of manually writing every rule, we provide examples and let the algorithm learn useful patterns.

Core abstraction

$$f_\theta(x) \approx y$$

Step 1: Input

x contains measurable features such as hours studied, attendance and previous score.

Step 2: Model

f_theta is a model with parameters theta learned from data.

Step 3: Output

y is the desired prediction such as final score or pass/no-pass.

Key shift from Part 2

Symbolic AI writes explicit rules. Machine learning estimates patterns from data and evaluates them statistically.

ML learns functions from examples instead of hand-coded rules

Universe of ML and Generative AI Models

Problem types, model families and examples



Supervised Learning

Regression
- Linear/Polynomial/Ridge/Lasso
- Bayesian Linear Regression
- Decision Tree / Random Forest
- Gradient Boosting / XGBoost
- k-NN Regressor / SVR / MLP

Classification
- Logistic/Softmax Regression
- Naive Bayes / Bayesian models
- k-NN / SVM / Perceptron
- Tree / Forest / Boosting

Unsupervised Learning

Clustering
- k-Means / Hierarchical
- DBSCAN / GMM

Dimensionality Reduction
- PCA / SVD / t-SNE / UMAP

Density and Anomaly
- KDE / Isolation Forest
- One-Class SVM / LOF

Association Rules
- Apriori / FP-Growth

Ensemble and RL

Ensemble Learning
- Bagging / Random Forest
- Boosting: AdaBoost, GBDT
- XGBoost / LightGBM / CatBoost
- Stacking / Blending

Reinforcement Learning
- Q-Learning / SARSA
- DQN / Policy Gradient
- Actor-Critic / PPO

Deep Learning and GenAI

Deep Learning
- MLP / CNN / RNN / LSTM / GRU
- Autoencoders / GNNs
- Transformers / Attention

Classical Generative
- Naive Bayes / HMM / GMM

Modern Generative AI
- VAE / GAN / Diffusion
- GPT-style LLMs / T5 / BART
- Multimodal models / Agents

Important distinction: k-NN is supervised for classification/regression. k-Means is unsupervised clustering.

Generative AI is not separate from ML; it is a major family of generative models, often built with deep learning.

Expanded taxonomy with model examples added per review feedback

A Common Dataset

One table reused across Parts 3, 4 and 5

We will use one small student-performance dataset so students can see how different ML algorithms solve the same problem in different ways.

Student	Hours	Attend.	Prev	Final	Pass?
A	2	60	45	50	No
B	4	75	60	67	Yes
C	5	85	70	78	Yes
D	7	92	82	90	Yes
E	3	68	52	58	No

Two targets

Regression target: Final Score. Classification target: Pass? = Yes or No.

Common sample dataset for the module

Features, Labels, X and y

How a table becomes mathematics

Step 1: Choose features

x = [hours, attendance, previous_score].

Step 2: Choose target

Regression y = final_score; classification y = pass/no-pass.

Step 3: Build X and y

Rows become samples; columns become features.

Matrix view

```
X = [[2,60,45], [4,75,60], [5,85,70], [7,92,82], [3,68,52]]
y_reg = [50,67,78,90,58]
y_cls = [0,1,1,1,0]     where 1 = Pass and 0 = No
```

ML converts messy real-world tables into numerical vectors and matrices.

Matrix view is foundational for modern ML

Matrix/Tensor View and Hardware Speed

Why ML is written in batches

Matrix notation lets us process many examples at once. Modern ML frameworks map these operations to optimized tensor kernels on GPUs and TPUs.

Step 1: Loop style

Predict for A, then B, then C, then D, then E. This is simple but slow at large scale.

Step 2: Batch style

Compute yhat = X beta once and obtain all predictions together.

Tiny example

[[1,2],[1,4],[1,5],[1,7],[1,3]] x [34.15,8.20]^T = predictions

Why faster?

- Matrix multiplication uses parallel hardware.
- Large batches reduce overhead.

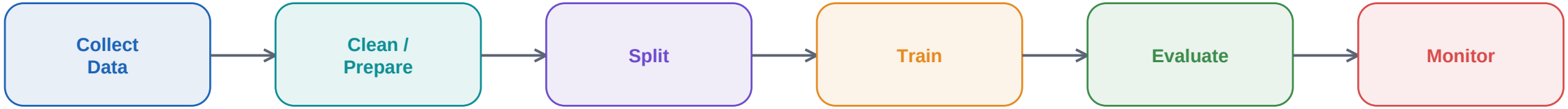
Deep learning connection

- Images, text embeddings and LLM activations are tensors.
- GPU/TPU cores accelerate tensor math.

Matrix example shows why batch computation matters

ML Pipeline and Data Split

From raw data to evaluated model





Step 1: Train

Fit parameters using training data.

Step 2: Validate

Choose model settings without touching test data.

Step 3: Test

Estimate final generalization performance.

Data splitting avoids overly optimistic evaluation

Loss, Cost, Risk and Generalization

How models are judged

Step 1: Loss

Loss is error on one example: L_i = (y_i - yhat_i)^2 for regression.

Step 2: Cost / Objective

Cost is what the algorithm minimizes. It aggregates losses and may include penalties.

Step 3: Empirical Risk

Average training loss: R_emp = (1/n) sum_i L_i.

Weak model: avg loss 193.4 | Learned line: avg loss 2.28

Step 4: Generalization

A model must perform well on unseen data, not only training examples.

Step 5: Bias vs variance

Underfitting = high bias. Overfitting = high variance. Good models balance both.

Detailed algorithms continue in Part 4

PART 4A - REVISED

Linear Regression
Four Teaching Views

Matrix, Newton/second-derivative, Gradient Descent and Bayesian derivations

1. Matrix Normal Equation

2. Newton / Second Derivative

3. Gradient Descent

4. Bayesian View

REVISION

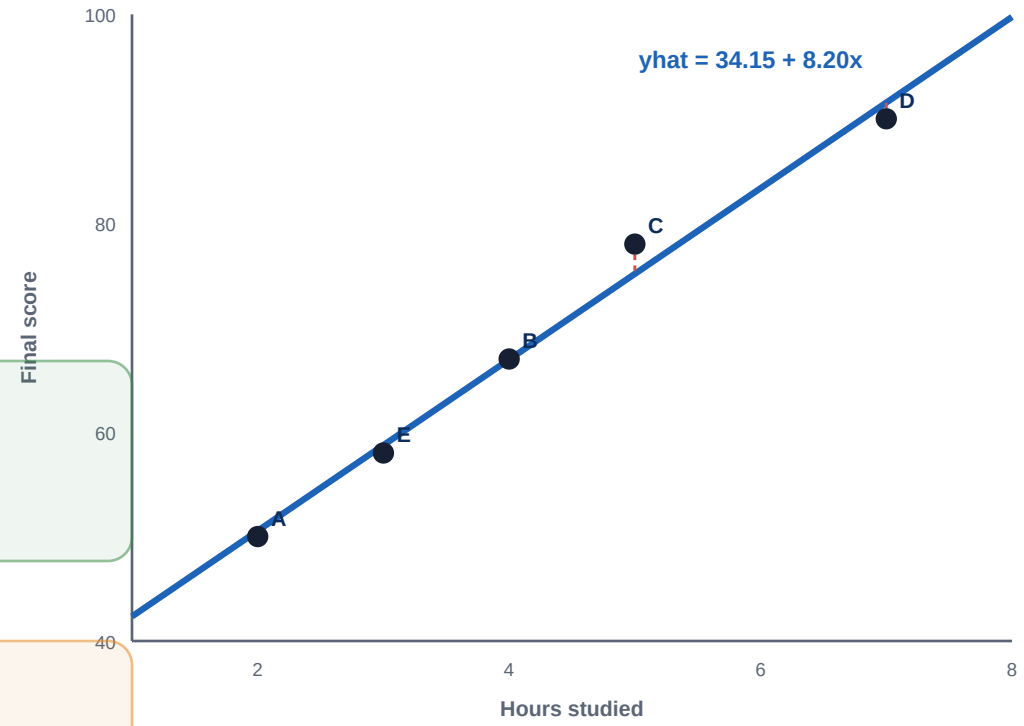
Expanded Way 2 and Way 4 with assumptions, beta meaning, formulas, numeric substitution and figures.

Only linear regression; other algorithms will follow after review

Linear Regression Starts from One Question

Can study hours predict final score?

Student	Hours x	Final y	Goal
A	2	50	predict y
B	4	67	predict y
C	5	78	predict y
D	7	90	predict y
E	3	58	predict y



Step 1: Input

Use x = hours studied because it is easy to visualize.

Step 2: Output

Use y = final score because it is numeric.

Step 3: Goal

Find a line that makes vertical errors between actual y and predicted yhat as small as possible.

Red dashed segments are residuals: actual score minus predicted score

Model, Residual, Loss and Cost

This is the common base for all four derivations

Model

$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

Residual

$$e_i = y_i - \hat{y}_i$$

Cost

$$J = \sum_i e_i^2$$

Student	x	y	yhat	resid.	loss
A	2	50	50.55	-0.55	0.30
B	4	67	66.95	0.05	0.00
C	5	78	75.15	2.85	8.12
D	7	90	91.55	-1.55	2.40
E	3	58	58.75	-0.75	0.56

Interpretation

The fitted line $\hat{y}=34.15+8.20x$ has small residuals. Total squared error is about 11.4, so MSE is about 2.28.

Numbers rounded for classroom readability

What is beta?

Before solving, students must know what the parameters mean

beta0: intercept

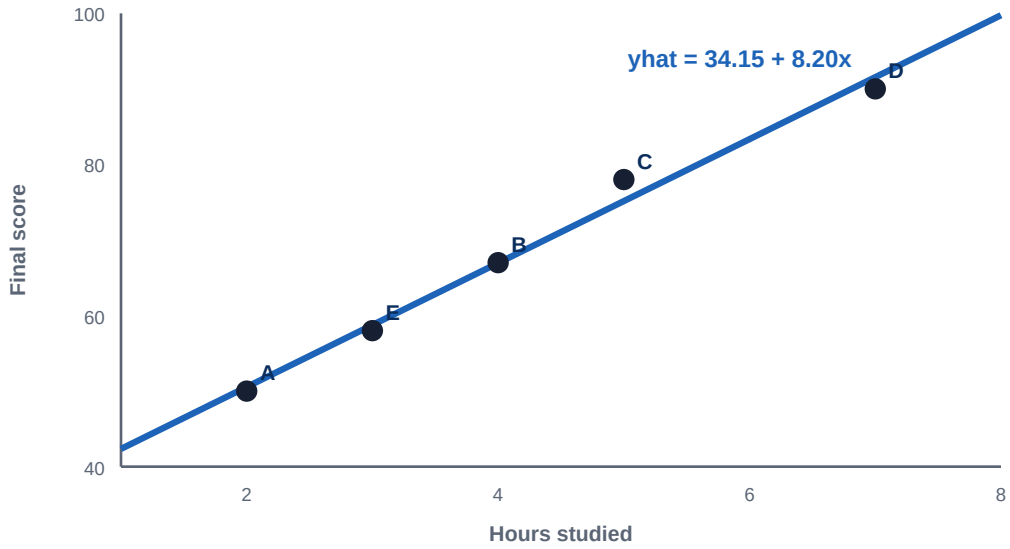
Predicted score when x=0. It shifts the line up or down. In our fitted line, beta0=34.15.

beta1: slope

Expected score increase for one additional study hour. In our fitted line, beta1=8.20 marks/hour.

Parameter vector

```
beta = [beta0,
     beta1]^T
```



All four methods solve for the same beta vector

Way 1 - Matrix Method: Build X, beta and y

The line becomes one matrix equation

Step 1: Add intercept

Each row is [1, x_i], not only x_i. The 1 learns beta0.

Step 2: Stack examples

All five students become one design matrix X.

Step 3: Predict all at once

yhat = X beta gives all five predictions in one operation.

Design matrix X

```
[[1,2],
 [1,4],
 [1,5],
 [1,7],
 [1,3]]
```

Parameter vector

```
beta = [beta0,
        beta1]^T
```

Prediction vector

```
[50.55,
 66.95,
 75.15,
 91.55,
 58.75]^T
```

Matrix prediction

$$\hat{y} = X\beta$$

Matrix notation is the foundation for multi-feature regression and tensor computation

Way 1 - Matrix Method: Solve the Normal Equation

Derivation and substitution

Step 1: Start from cost

J(beta)=||y-Xbeta||^2.

Step 2: Differentiate

Set gradient to zero: X^T X beta = X^T y.

Step 3: Solve

beta=(X^T X)^(-1)X^T y.

Dataset matrix products

```
X^T X = [[5,21],[21,103]]
X^T y = [343,1562]^T
```

Final substitution

```
beta = [[5,21],[21,103]]^-1
       x [343,1562]^T
```

Answer

beta = [34.15, 8.20]^T -> yhat = 34.15 + 8.20x

Normal equation gives the exact least-squares solution when X^T X is invertible

Way 2 - Newton / Second-Derivative Method

Clarifying the idea first

Important correction

We do NOT set the double derivative to zero. At a minimum, the first derivative/gradient is zero. The second derivative/Hessian should be positive and tells the curvature of the cost surface.

Step 1: First derivative

Tells the slope/direction. Minimum occurs when slope becomes zero.

Step 2: Second derivative

Tells curvature: how quickly the slope changes.

Step 3: Newton step

Uses slope and curvature to jump toward the minimum.

One-parameter Newton formula

$$\text{theta_new} = \text{theta_old} - J'(\text{theta_old}) / J''(\text{theta_old})$$

Newton-Raphson uses first and second derivative; gradient=0 is the optimality condition

Way 2 - Newton Method for Linear Regression

Vector form with beta0 and beta1

For two parameters beta=[beta0,beta1]^T, the one-variable derivative becomes a gradient vector, and the second derivative becomes a Hessian matrix.

Step 1: Cost

J(beta)=1/2 ||y-Xbeta||^2. The 1/2 makes derivatives cleaner.

Step 2: Gradient

grad J = X^T X beta - X^T y.

Step 3: Hessian

H = X^T X. For squared loss, this is constant.

Newton update

$$\text{beta_new} = \text{beta_old} - H^{-1} \nabla J(\text{beta_old})$$

Why Newton is special here

The squared-error cost is a quadratic bowl. Newton uses the bowl curvature, so for linear regression it reaches the exact bottom in one step.

Vector Newton method uses gradient and Hessian

Way 2 - Newton Method: Numeric Substitution

Start from beta=[0,0] and calculate

Step 1: Start beta

beta_old=[0,0]. Therefore predictions are all zero.

Step 2: Gradient

grad = X^T X beta - X^T y = -X^T y = [-343,-1562].

Step 3: Hessian

H = X^T X = [[5,21],[21,103]].

Newton update

```
beta_new = [0,0] - H^-1[-343,-1562]
         = H^-1[343,1562]
```

Result

```
beta_new = [34.15, 8.20]
line = 34.15 + 8.20x
```

Classroom sentence

Newton gives the same answer as the normal equation because both solve gradient=0.

Numeric Newton calculation using the sample dataset

Way 2 - Newton Method: Figure

Use curvature to estimate where the minimum is



What the orange line shows

Newton builds a local quadratic/curvature approximation and jumps to a better parameter value.

Why not always use Newton?

It needs the Hessian inverse. For huge models, computing and inverting Hessian is expensive, so gradient descent is often preferred.

Figure is conceptual: Newton step from slope and curvature

Way 3 - Gradient Descent Method

Take many small steps downhill

Step 1: Initialize

Start beta=[0,0]. Then yhat=0 for every student.

Step 2: Compute gradient

Using MSE, gradient at beta=0 is about [-137.2,-624.8].

Step 3: Update

beta <- beta - eta*gradient. If eta=0.001, beta becomes [0.137,0.625].

Update rule

$$\text{beta}^{(t+1)} = \text{beta}^{(t)} - \text{eta} * \text{gradient } J(\text{beta}^{(t)})$$

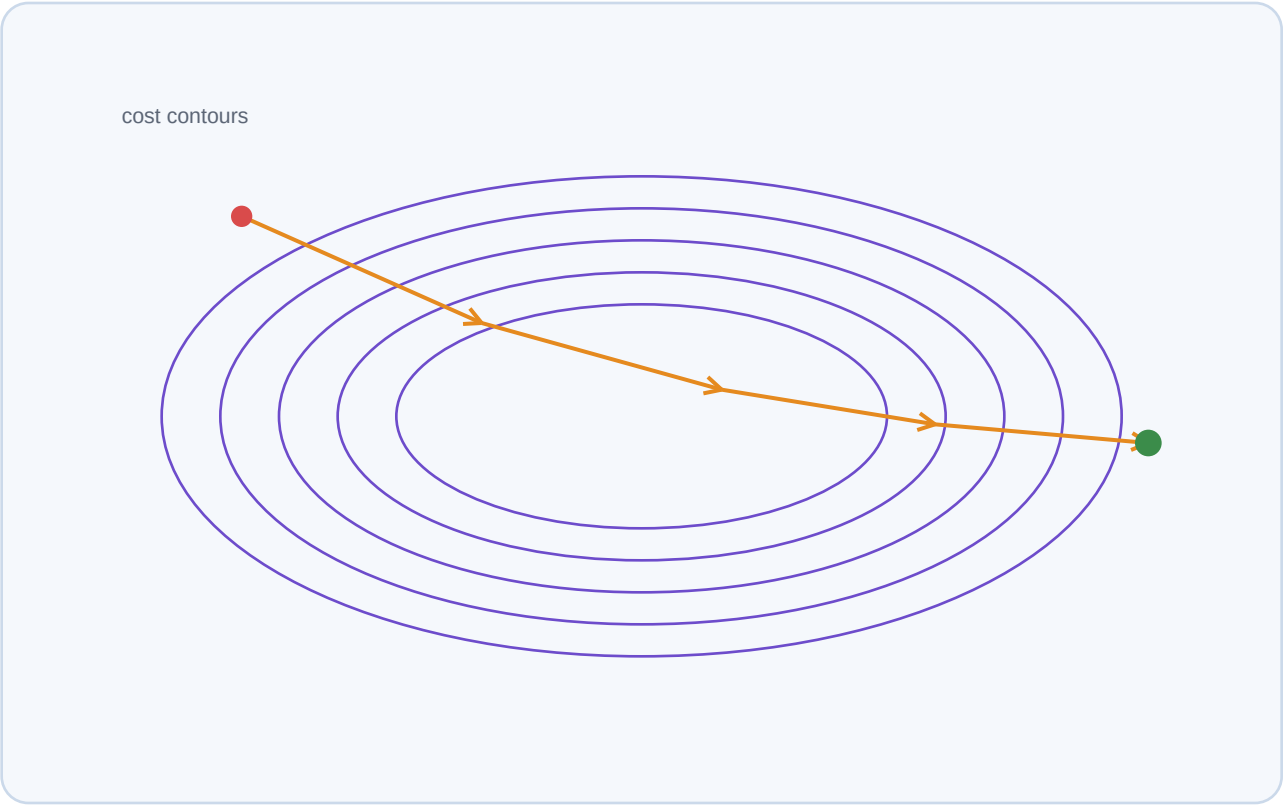
Why use gradient descent?

The normal equation is fine for this tiny example. Gradient descent is the method that scales to huge datasets, nonlinear models and deep learning.

Gradient descent is slower than Newton here, but more general

Way 3 - Gradient Descent Method: Figure

Move through the cost surface toward the minimum



Step-by-step story

- Start with a bad line.
- Compute direction of steepest increase.
- Move opposite that direction.
- Repeat until cost stops decreasing.

Result

After enough iterations, beta approaches [34.15, 8.20], the same least-squares solution.

Figure: contour lines of cost; arrows show gradient descent path

Way 4 - Bayesian Method: Assumptions

What exactly are we assuming?

Step 1: Linear relation

y_i = beta0 + beta1 x_i + epsilon_i.

Step 2: Noise assumption

epsilon_i is Gaussian noise: epsilon_i ~ N(0, sigma^2).

Step 3: Prior on beta

Before data, beta is uncertain: beta ~ N(mu0, Sigma0).

Meaning of beta in Bayesian regression

beta is not treated as one fixed unknown number. It is treated as a random variable with a probability distribution that becomes sharper after seeing data.

Likelihood

$$y \mid X, \beta, \sigma^2 \sim N(X\beta, \sigma^2 I)$$

Bayesian regression requires a likelihood and a prior

Way 4 - Bayesian Method: Prior, Likelihood, Posterior

The learning rule is Bayes theorem

Step 1: Prior

What we believe before data: p(beta). Example: beta ~ N([0,0], tau^2 I).

Step 2: Likelihood

How likely the data are for a given beta: p(y | X,beta).

Step 3: Posterior

Updated belief after data: p(beta | D).

Bayes update

$$p(\beta \mid D) \propto p(y \mid X,\beta)\, p(\beta)$$

Teaching sentence

OLS asks: which beta minimizes squared error? Bayesian regression asks: after seeing the data, which beta values are probable?

Bayesian regression returns a distribution, not only one beta

Way 4 - Bayesian Method: How to Solve beta

Closed-form posterior for Gaussian prior and noise

Assume beta ~ N(mu0, Sigma0) and y|X,beta ~ N(Xbeta, sigma^2 I). Then the posterior is also Gaussian.

Posterior covariance

$$\Sigma_N = (\Sigma_0^{-1} + (1/\sigma^2) X^T X)^{-1}$$

Posterior mean

$$\mu_N = \Sigma_N (\Sigma_0^{-1} \mu_0 + (1/\sigma^2) X^T y)$$

How to read this

X^T X and X^T y are the same dataset summaries used in the normal equation. The prior adds Sigma_0^-1 terms. With a very broad prior, posterior mean becomes almost the same as OLS.

Conjugate Bayesian linear regression formula

Way 4 - Bayesian Method: Numeric Example

Use a broad prior so data mostly controls the result

Step 1: Choose prior

mu0=[0,0], Sigma0=10000 I. This says beta is uncertain and prior is broad.

Step 2: Choose noise

Assume sigma^2=25. This means score noise standard deviation is 5 marks.

Step 3: Use data summaries

X^T X=[[5,21],[21,103]], X^T y=[343,1562].

Posterior mean calculation result

`mu_N = [34.04, 8.23] approximately`

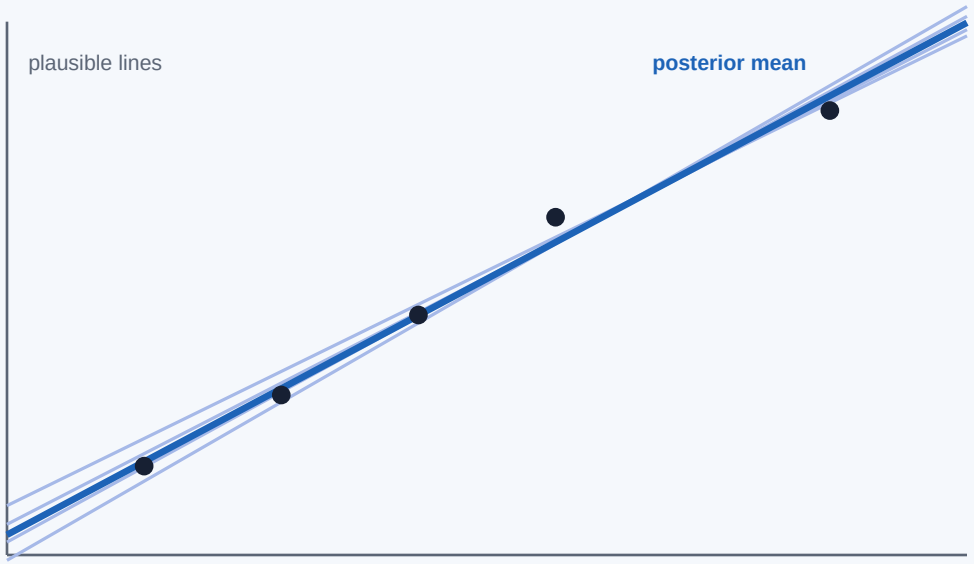
Interpretation

Bayesian answer is close to OLS [34.15,8.20], but Bayesian method also provides uncertainty through Sigma_N.

Numbers use broad Gaussian prior and sigma=5 marks

Way 4 - Bayesian Method: Figure

Multiple plausible lines after observing data



What the figure means

The dark line is posterior mean. Lighter lines are plausible beta values still supported by limited data.

Why this matters

With only five students, we should not be overconfident. Bayesian regression explicitly shows uncertainty.

Figure: posterior uncertainty over regression lines

Four Views of the Same Linear Regression Problem

Same answer, different teaching purpose

View	Starts with	Learning rule	Strength	Teaching message
Matrix	yhat = X beta	beta=(X^T X)^-1 X^T y	Compact and scalable	Many examples solved together
Newton / Hessian	J=1/2||y-Xbeta||^2	beta <- beta-H^-1 grad	Fast for quadratic loss	Uses curvature to jump to minimum
Gradient Descent	initial beta guess	beta <- beta-eta grad	General and scalable	Foundation of deep learning
Bayesian	prior + likelihood	posterior p(beta|D)	Uncertainty aware	Best line plus confidence

All four explain the same fitted line: yhat = 34.15 + 8.20 x_hours

End of Part 4A linear regression teaching edition

Part 4B - Logistic Regression

1 slide

Step 1: Encode target

Pass=1 and No=0. Use hours as the feature.

Step 2: Linear score

z = beta0 + beta1 x. Example: z = -6.5 + 1.8x.

Step 3: Probability

p = 1/(1+exp(-z)). For B, x=4 gives p≈0.668.

Step 4: Loss

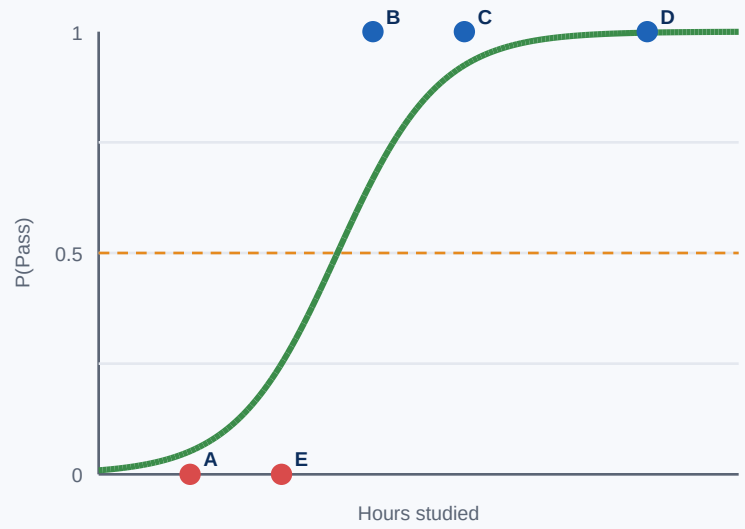
Cross-entropy: L=-[y log(p)+(1-y)log(1-p)].

Stud	x	y	p	Pred
A	2	0	0.052	No
E	3	0	0.250	No
B	4	1	0.668	Pass
C	5	1	0.924	Pass
D	7	1	0.998	Pass

Key equations

p = sigma(z)

sigma(z)=1/(1+e^-z)



Decision rule: if P(Pass) >= 0.5, predict Pass

No eta unless trained with gradient descent

Support Vector Machine

1 slide

Step 1: Encode labels

No=-1 and Pass=+1.

Step 2: Score function

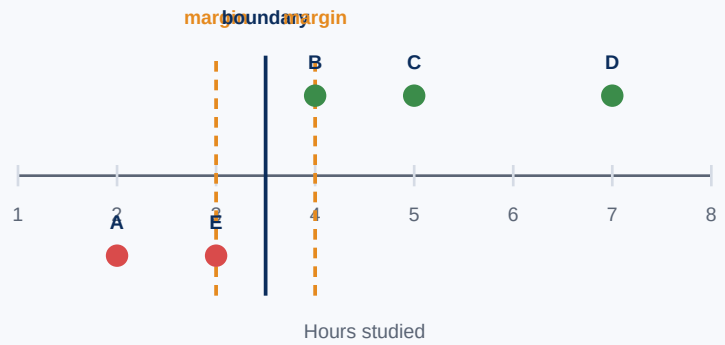
score=wx+b. Boundary is near x=3.5.

Step 3: Margin rule

Correct confident points satisfy y_i(wx_i+b)>=1.

Step 4: Hinge loss

L_i=max(0,1-y_i score_i).



Stud	x	y	score	hinge
A	2	-1	-3	0
E	3	-1	-1	0
B	4	1	+1	0
C	5	1	+3	0
D	7	1	+7	0

Objective

min 0.5||w||^2
+ C sum hinge

No eta in objective; iterative solvers may use step size internally.

Support vectors are closest points to the boundary

Boosting with a Regression Tree

Slide 1 of 2 - algebra and residual table

Step 1: Initialize

F0 = mean(y) = 68.6.

Step 2: Residuals

r_i = y_i - F0(x_i). These are the mistakes.

Step 3: Weak learner

Fit a tiny regression tree h1(x) to residuals.

Step 4: Update

F1 = F0 + eta h1. eta is learning rate.

S	y	F0	resid	h1	F1 eta=.5
A	50	68.6	-18.6	-10.27	63.47
B	67	68.6	-1.6	-10.27	63.47
C	78	68.6	9.4	15.4	76.3
D	90	68.6	21.4	15.4	76.3
E	58	68.6	-10.6	-10.27	63.47

Core idea

new model = old model + small correction

What is eta?

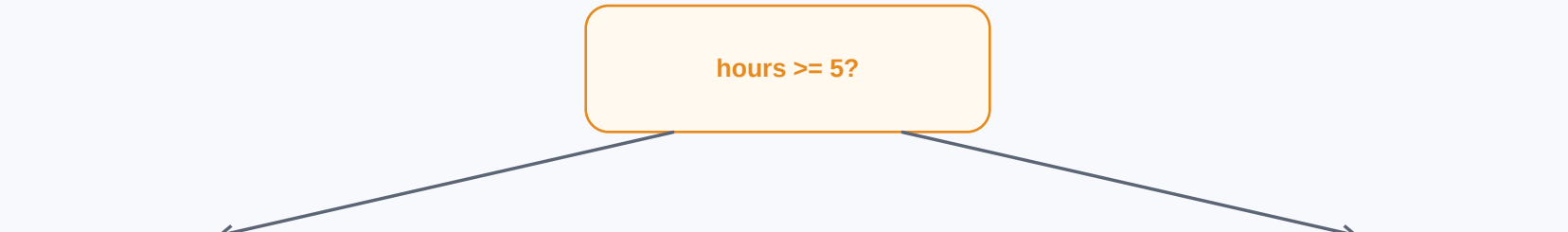
eta is the shrinkage or learning-rate factor. Typical values: 0.01, 0.05, 0.1. Smaller eta usually needs more trees but often generalizes better.

Choose eta using a validation set or cross-validation.

Algebra and eta explanation separated from figure

Boosting with a Regression Tree

Slide 2 of 2 - tree figure with no overlaps



Formula is intentionally below the child nodes to avoid overlap.

Update after first tree

$$F_1(x) = F_0(x) + \eta \cdot h_1(x)$$

eta = 0.5 only for classroom arithmetic; real boosting often uses 0.01, 0.05 or 0.1.

v6: update formula is below child nodes with a clear gap

Classification Tree

Slide 1 of 2 - impurity math

Step 1: Parent node

3 Pass and 2 No. Parent Gini = 0.48.

Step 2: Candidate split

Try the question: hours >= 4?

Step 3: Child purity

Left A,E are No; right B,C,D are Pass.

Step 4: Best split

Weighted Gini becomes 0, so the split is perfect here.

Node	Pass	No	Gini
Parent	3	2	0.48
Left x<4	0	2	0
Right x>=4	3	0	0

Gini formula

$$Gini = 1 - \sum_k p_k^2$$

Weighted split cost

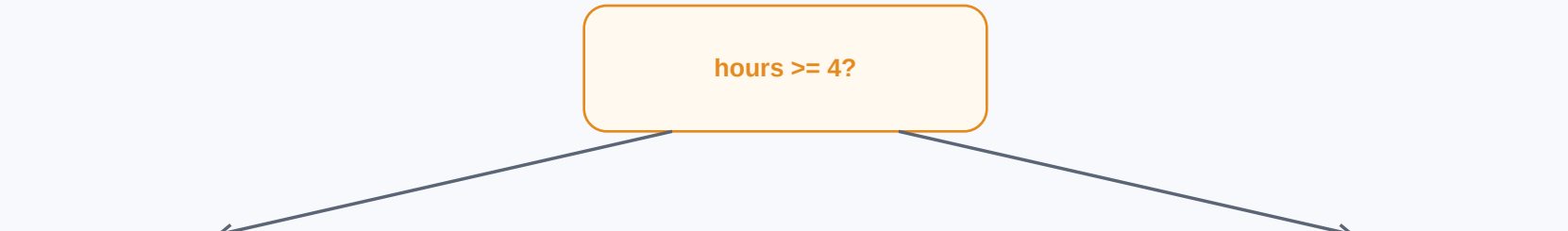
$$(2/5)*0 + (3/5)*0 = 0$$

No eta in a single tree; eta appears in boosting.

Gini math separated from figure

Classification Tree

Slide 2 of 2 - split figure with no overlaps



Formula is intentionally below the child nodes to avoid overlap.

Split quality

Weighted Gini = (2/5)*0 + (3/5)*0 = 0

This split makes both child nodes pure, so weighted Gini becomes zero.

v6: split-quality formula is below child nodes with a clear gap

Part 5 - Generative vs Discriminative Models

Three-slide wrap-up: probability view and GenAI bridge

Generative model

Learns how data could be generated. It models P(x,y) or P(x|y)P(y). It can often sample, generate or score data.

Discriminative model

Learns the boundary or direct conditional probability of the label given features. It focuses on prediction.

Generative view

P(x,y) = P(x|y)P(y)

Discriminative view

P(y|x) or decision boundary

Family	Learns	Typical output	Examples	Best for
Generative	P(x,y) or P(x|y)	new sample / likelihood	Naive Bayes, HMM, GMM, VAE, GAN, Diffusion, LLM	generation + uncertainty
Discriminative	P(y|x) or boundary	label / probability	Logistic Regression, SVM, discriminative neural classifier	prediction accuracy

Part 5 is intentionally limited to 3 slides as requested

Bayes, Naive Bayes and Sequence Models

Classical generative models before modern GenAI

Step 1: Bayes theorem

Use likelihood and prior to compute posterior: P(y|x).

Step 2: Naive Bayes

Assume features are conditionally independent given class.

Step 3: Sequence models

Markov chains and HMMs model sequences over time.

Bayes theorem

$$P(y|x) = P(x|y)P(y) / P(x)$$

Naive Bayes

$$P(y|x1,...,xd) \propto P(y) \prod P(xj|y)$$

Markov idea

$$P(s_t|s_1,...,s_\{t-1\}) \approx P(s_t|s_\{t-1\})$$

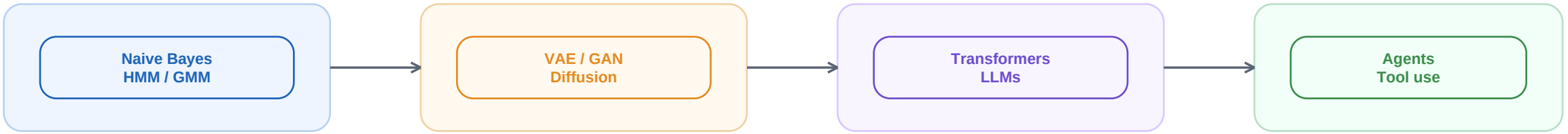
Using the student dataset

Naive Bayes would estimate P(Pass), P(No), and feature likelihoods such as P(hours group | Pass). HMM would be used only when data are sequential, for example student progress week by week.

Classical generative models use probability assumptions explicitly

Bridge to Modern Generative AI

From probability models to LLMs, diffusion and agents



LLM next-token learning

P(token_t | tokens_<t)

Diffusion intuition

learn to denoise noise -> data

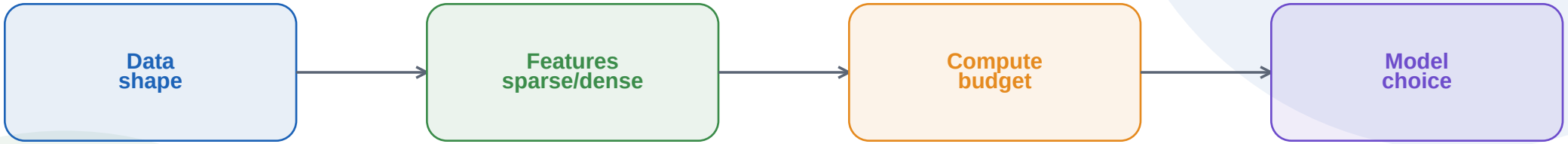
Model type	What it learns	What it generates	Example
LLM	next-token probability	text/code/reasoning trace	GPT-style model
Diffusion	reverse denoising process	images/audio/video	image generation
Agentic AI	model + tools + memory + plan	actions/workflows	research assistant/SOC copilot

This closes Module 1 and prepares students for transformer and GenAI architecture later

MODEL SELECTION GUIDE

Which Model to Use When?

Data sparsity, feature sparsity, compute, complexity and generative vs discriminative choice



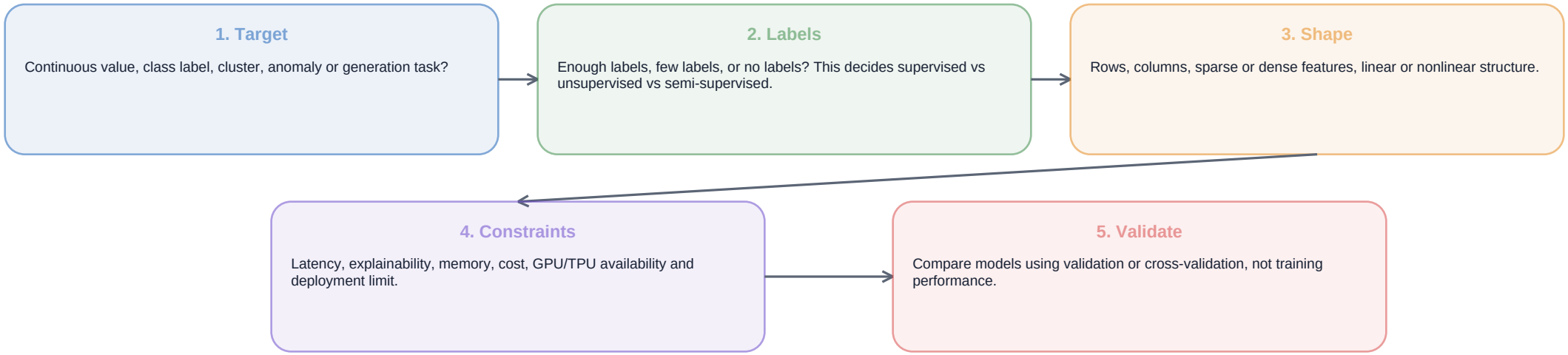
Core rule

Start with the simplest model that matches the data and constraint. Move to more complex models only when validation performance, feature interactions or nonlinear structure justify it.

Use this as the decision slide after Part 4/Part 5

Model Selection Workflow

A practical order for choosing models - fixed layout



Important teaching warning

- Do not choose models by popularity. Choose based on data and constraints.
- Always build a baseline first: linear, logistic, Naive Bayes or a simple tree.
- Use validation or cross-validation to select hyperparameters and final model.

Model choice = problem type + data size + feature shape + compute + business need

Fixed workflow slide: all text kept inside boxes

Model Choice by Data Size and Sparsity

What works when data is small, sparse or high-dimensional?

Sparse data vs sparse features: Sparse data = few training examples. Sparse features = many zeros in the feature matrix.

Situation	Examples	Prefer	Why	Avoid / caution	First baseline
Very small data	<1k rows	Naive Bayes, logistic, linear, simple tree	Lower variance and easier to regularize	Deep nets, huge boosting models	Naive Bayes / logistic
Few labels, many unlabeled	10%-20% labels	semi-supervised, embeddings + linear, self-training	Uses unlabeled structure	Fully supervised deep model from scratch	embedding + logistic
Sparse features	bag-of-words, one-hot	Naive Bayes, logistic, linear SVM	Efficient for high-dimensional sparse X	dense tree models without care	logistic / NB
Many features, fewer rows	d >> n	Lasso, Ridge, Elastic Net, linear SVM	Regularization controls overfit	unregularized linear, large trees	Ridge/Lasso
Dense tabular, nonlinear	finance, ops, student data	trees, random forest, boosting	Captures interactions and thresholds	deep nets unless data is large	tree/GBM
Very large unstructured	text, images, audio	deep learning, transformers, CNNs	Learns representations from raw data	manual features only	pretrained model

Sparse feature matrices are common in text, logs and one-hot encoded categorical data

Model Choice by Compute, Latency and Complexity

Approximate practical guidance, not exact benchmark

Rule of thumb: low latency or edge device -> linear/logistic/small tree; high accuracy tabular -> boosting; unstructured large-scale -> neural nets.

Model	Training cost	Prediction cost	Memory	Explainability	Good when	Watch out
Linear / Ridge	low	very low O(d)	low	high	fast baseline, dense/sparse data	linear pattern only
Logistic Reg.	low-medium	very low O(d)	low	high	binary/multiclass with many features	linear boundary
Naive Bayes	very low	very low	very low	medium	text, sparse data, small data	independence assumption
Decision Tree	medium	low O(depth)	medium	high	rules, nonlinear tabular	overfits
Random Forest	medium-high	medium	high	medium	stable tree model	many trees use memory
Boosting/XGBoost	high	medium	medium-high	medium	best tabular accuracy	tuning and sequential training
SVM Linear	medium	low O(d)	medium	medium	high-dimensional sparse data	calibration
SVM Kernel	high O(n^2+)	medium-high	high	low-medium	small/medium nonlinear data	does not scale well
Neural Nets	high	medium-high	high	low	large unstructured data	needs GPU/data/tuning

n = rows, d = features; actual cost depends on implementation and hardware

Choose by Problem Type

Start from the target variable

The target decides the model family first; data/compute constraints decide the specific model.

Problem type	Target / label	Best starting models	Advanced models	Evaluation metric
Regression	continuous y	Linear/Ridge/Lasso, regression tree	Random forest regressor, gradient boosting, neural regressor	RMSE, MAE, R2
Classification	class label	Logistic regression, Naive Bayes, tree	SVM, random forest, gradient boosting, neural classifier	F1, AUC, precision, recall
Clustering	no labels	k-Means, hierarchical	DBSCAN, GMM, spectral clustering	silhouette, domain review
Anomaly detection	rare abnormal cases	Isolation Forest, One-Class SVM	autoencoder, density models	precision@k, recall, alert quality
Ranking	ordered relevance	linear ranker, tree ranker	LambdaMART, neural ranker	NDCG, MAP, MRR
Generation	create text/image/code	language model, VAE/GAN/diffusion	LLM, diffusion, multimodal model	human eval, task metrics, safety

Problem type first, algorithm second

Choose by Feature Shape

Sparse, dense, categorical, time-series and unstructured data

High-dimensional does not automatically mean deep learning. For sparse tabular/text features, linear models are often strong and cheap.

Feature shape	Example	Prefer	Why	Notes
Sparse high-dimensional	text TF-IDF, logs	Naive Bayes, logistic, linear SVM	They handle many zero-valued features efficiently	Use regularization
Many categorical features	one-hot encoded business data	boosting, CatBoost, logistic	Boosting captures nonlinear category interactions	CatBoost handles categoricals well
Dense numerical tabular	scores, measurements	linear/logistic baseline, trees, boosting	Trees find thresholds and interactions	Scale features for linear/SVM
Time-series	sensor, stock, usage over time	ARIMA baseline, RNN/LSTM, Transformer	Temporal order matters	Use time-based split
Images	pixels	CNN, ViT, diffusion	Learns spatial representation	Use pretrained models
Text/code	tokens	Transformers, LLMs	Learns semantic representation	Use embeddings/finetuning/RAG

Feature engineering and representation often matter more than model name

When to Use Generative vs Discriminative Models

The decision depends on the question

Use Generative Models When

- You need to generate text, image, audio, code or synthetic data.
- You need likelihood/anomaly scoring: how likely is this data?
- Labels are limited but distribution modeling helps.
- You need probabilistic reasoning with priors and likelihoods.

Use Discriminative Models When

- Your main goal is accurate prediction of y from x.
- You have labeled data.
- You need a decision boundary or direct P(y|x).
- You want efficient supervised classification/regression.

Question	Prefer	Learns	Examples	Why
Will this student pass?	Discriminative	P(y|x) or boundary	logistic, SVM, neural classifier	directly optimizes label pre
Generate a realistic student profile	Generative	P(x) or P(x,y)	VAE, GAN, diffusion, LLM	models data distributi
Classify with very little data	Often generative baseline	P(x|y)P(y)	Naive Bayes	strong assumptions reduce d
Detect unusual behavior	Generative/density	likelihood of x	GMM, HMM, autoencoder	rare data gets low likelih

Generative models learn how data can arise; discriminative models focus on prediction

Fast Model Selection Cheat Sheet

Pick a sensible first model

Final teaching line: baseline first, validate second, complexity last.

Given situation	Use first	Then try	Reason	Avoid first
Few rows, simple numeric target	Ridge/linear regression	tree, boosting if nonlinear	low variance, interpretable	deep neural net
Few rows, classification	Naive Bayes/logistic	linear SVM, small tree	works with limited data	large forest/NN
Many sparse text features	Naive Bayes/logistic/linear SVM	transformer embeddings + classifier	efficient sparse math	kernel SVM
Dense tabular with interactions	decision tree	random forest, XGBoost/LightGBM/CatBoost	captures thresholds/interactions	plain linear only
Need explainability	linear/logistic/small tree	monotonic GBM + SHAP	auditable rules/coefficients	black-box deep net
Need low latency	linear/logistic/small tree	distilled model	cheap inference	large ensemble/LLM
Need generation	LLM/diffusion/VAE/GAN	finetuned/RAG/agentic system	task is generative	pure discriminative classifier
Need anomaly detection	Isolation Forest/GMM	autoencoder/HMM	models unusualness	standard classifier without labels

Use this as an interview-style answer and classroom summary

Losses, Hyperparameters and How to Choose Values

Model choice also means tuning

Eta / learning rate: controls update size. Typical boosting eta: 0.01-0.1; choose with validation and early stopping.

Model	Loss/cost	Important knobs	How to choose	Sparse/high-dim?
Linear/Ridge/Lasso	MSE/MAE	alpha/lambda regularization	cross-validation	yes, with regularization
Logistic	cross-entropy	C/lambda, threshold	CV + threshold by F1/recall	yes
SVM	hinge + margin	C, kernel, gamma	CV; linear for high-dim sparse	linear SVM yes
Decision Tree	Gini/entropy/MSE	max_depth, min_samples_leaf	validation/pruning	not ideal for huge sparse
Random Forest	tree loss + avg/vote	n_estimators, max_features	OOB/validation	works but memory heavy
Boosting	MSE/logloss/stage loss	eta, trees, depth	validation; early stopping	works well on tabular
Neural Net	task-specific	LR, layers, batch size	validation, schedules, Adam	depends on architecture

Loss defines the goal; hyperparameters control model complexity and learning behavior